



Samueli
School of Engineering

CS-M151B

Computer Systems Architecture

Blaise Tine
UCLA Computer Science

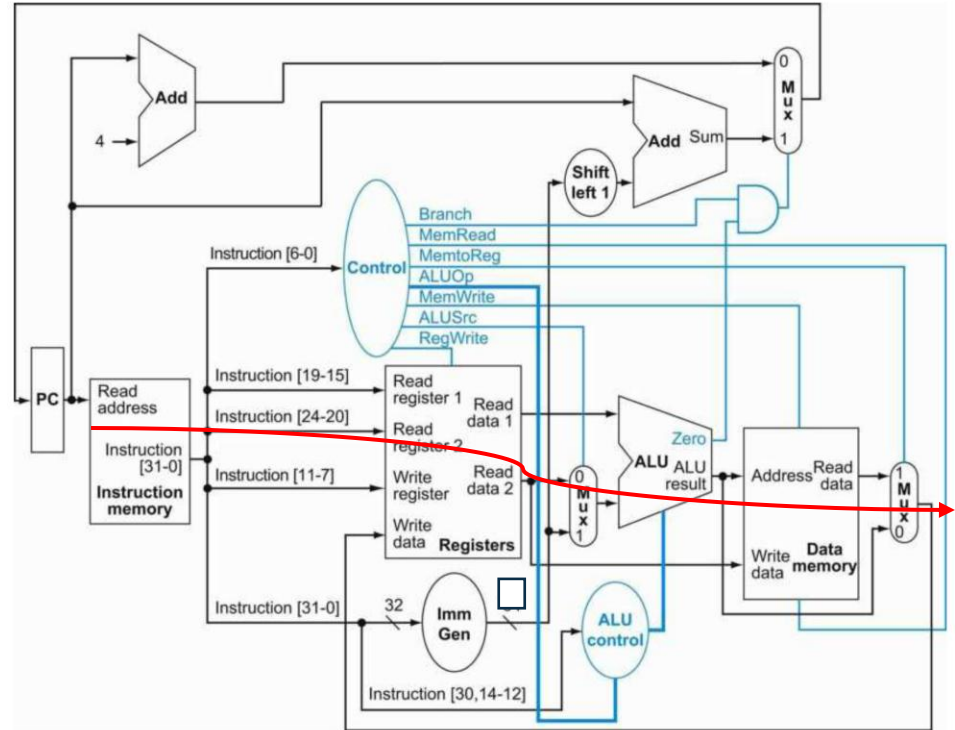
Single-Cycle Processor

Single-cycle Processor Latency:

- How this affects CPU latency?
- Longest delay determines the clock speed
- CPU clock affects performance

$$\text{CPU Time} = \text{InstCount} * \text{CPI} * \text{CycleTime}$$

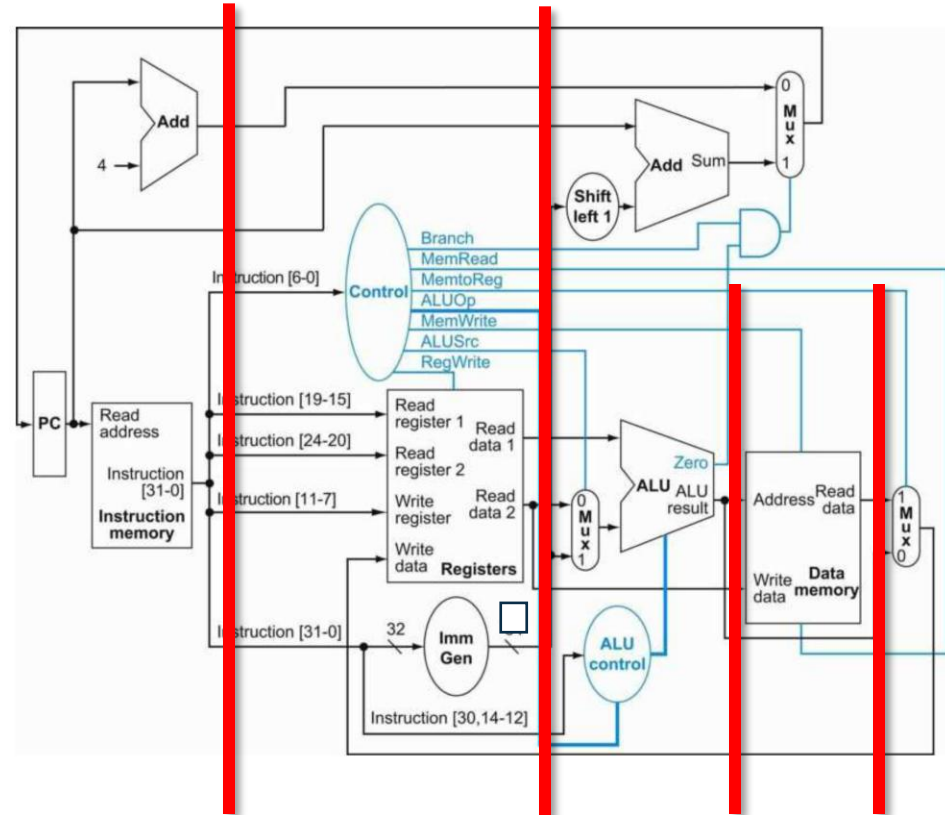
How to reduce propagation delay?



Single-Cycle Processor

Reducing Propagation Delay

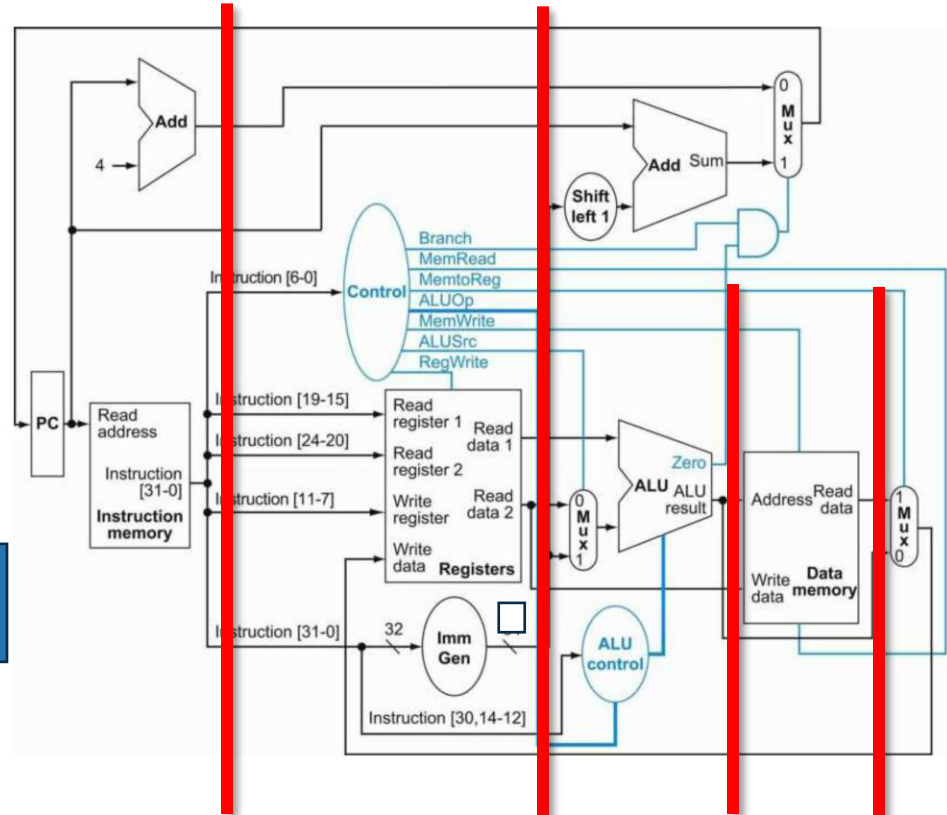
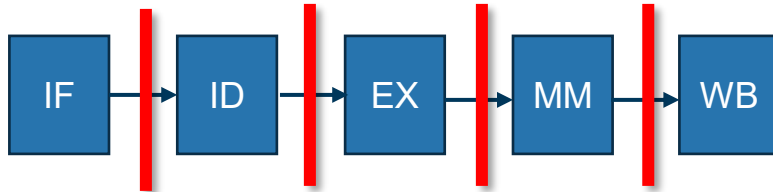
- Split the datapath into smaller chunks
- Insert registers in between
- Update control logic
 - Control registers inputs/outputs



Multi-Cycle Processor

Change to multi-cycle architecture

- 5 stages
 - IF: Instruction Fetch
 - ID: Register access
 - EX: ALU operation
 - MEM: Memory access
 - WB: Write back

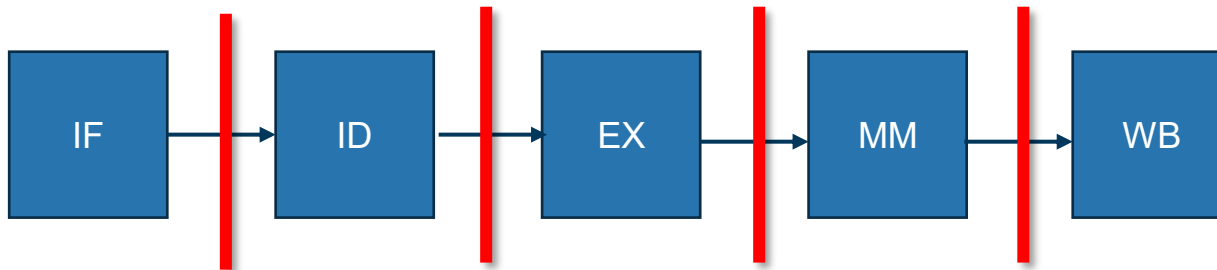


Multi-Cycle Processor

Improved Propagation Delay

- Longest propagation delay?
 - $\max(\text{IF}, \text{ID}, \text{EX}, \text{MEM}, \text{WB})$
 - Should be smaller

How is the overall performance impacted?

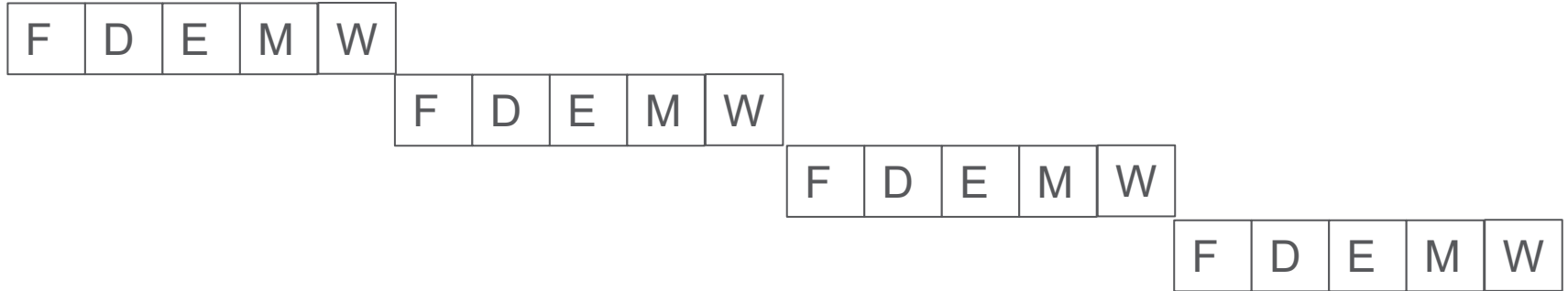


Multi-Cycle Processor

Multi-cycle Processor Performance

- **Pros:** Cycle Time is reduced
- **Cons:** CPI is increased: a single instruction takes multiple cycles to complete

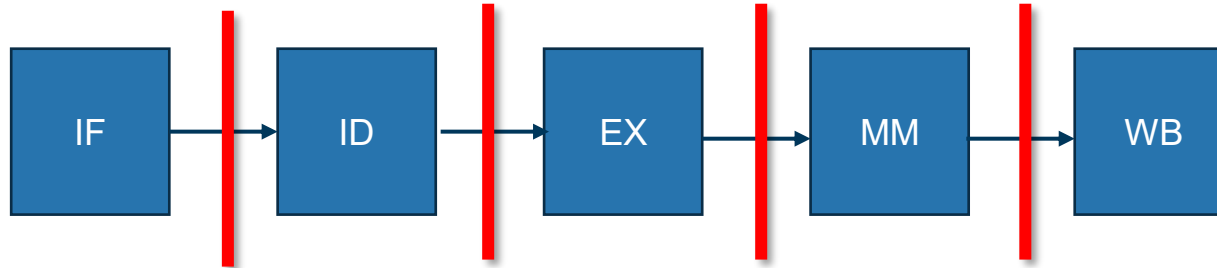
$$\text{CPU Time} = \text{InstCount} * \text{CPI} * \text{CycleTime}$$



Pipelined Processor

Observations?

- **H/W resources are under utilized**
- When an instruction is stage WB
- IF, ID, E, and MEM stages are idle (do nothing)

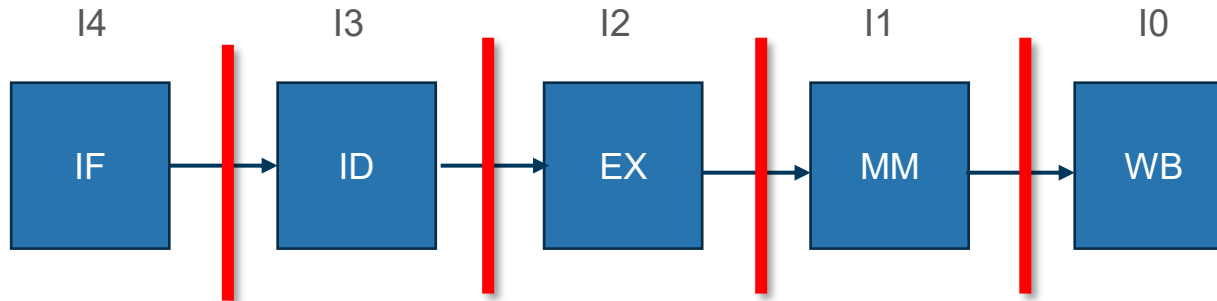


Pipelined Processor

Solution

- Fetch a new instruction each cycle

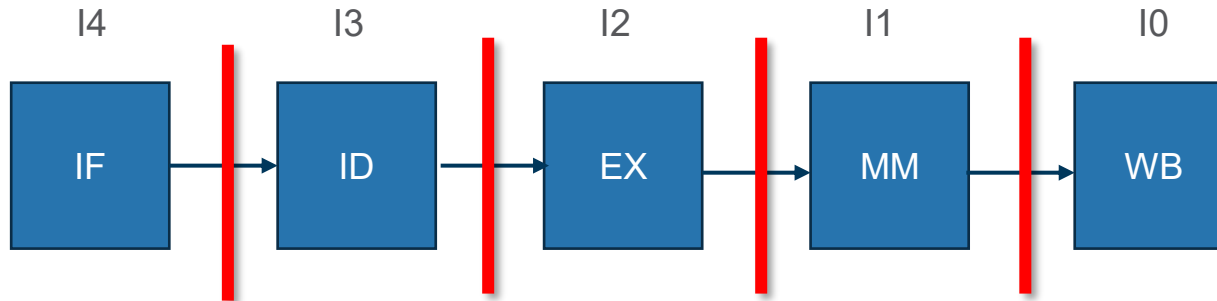
e.g: Executing instructions sequence: I0, I1, I2, I3, I4



Pipelined Processor

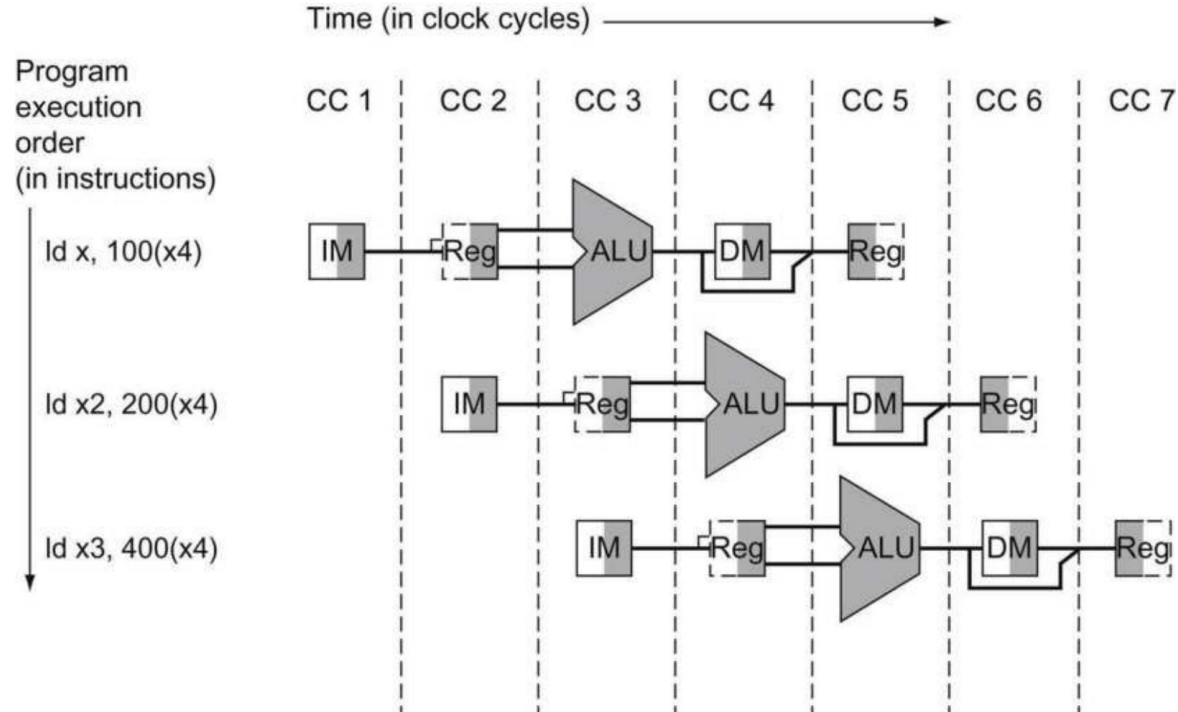
How it work?

- The PC should be incremented each cycle
- Online instructions are kept in pipeline registers



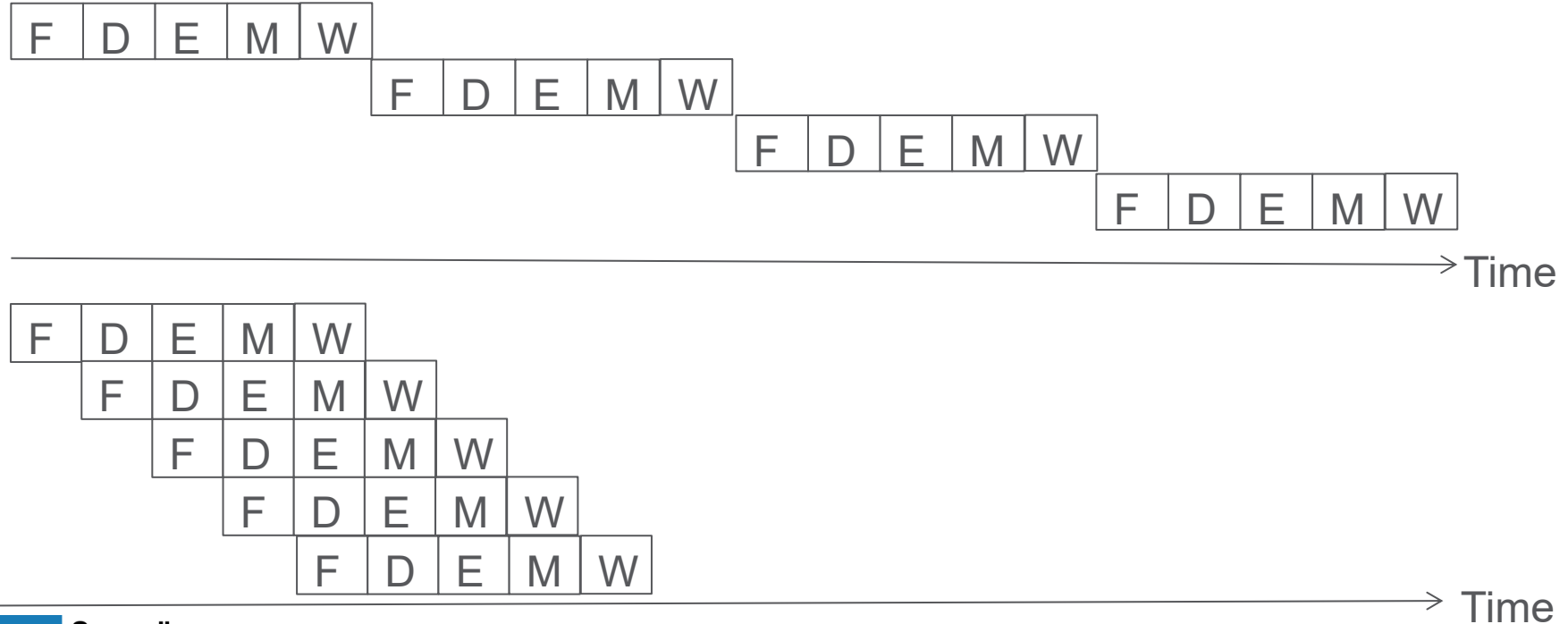
Pipelined Processor

Resource utilization



Pipelined Processor

Multi-cycle vs Pipelined performance



Pipelined Processor

Multi-cycle vs Pipelined performance

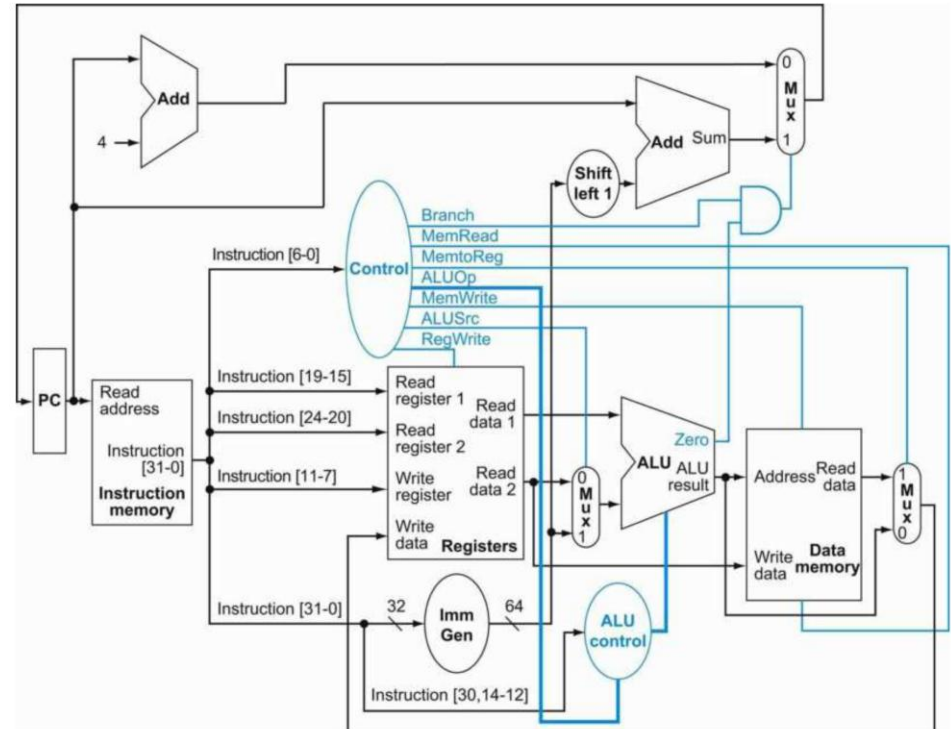
- The CPI is reduced
- Ideal pipelined CPI is 1
 - One instruction complete each cycle

$$\text{CPU Time} = \text{InstCount} * \text{CPI} * \text{CycleTime}$$

Exercise: Single-Cycle vs Pipeline

- Calculate cycle time (CT)

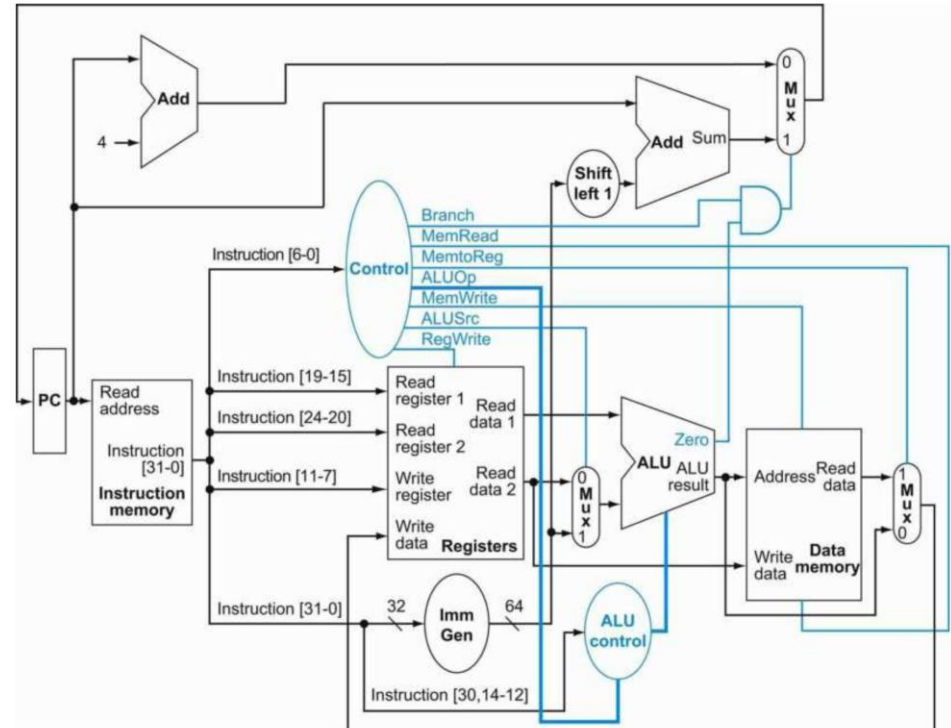
Unit	Latency (ps)
Instruction Fetch	400
Register Access	300
ALU	200
Memory Access	400



Exercise: Single-Cycle vs Pipeline

- Single-Cycle Datapath

Instr	Data Path
R/I-type	IF+RR+ALU+RW
LW	IF+RR+ALU+MR+RW
SW	IF+RR+ALU+MW
BEQ	IF+RR+ALU



Exercise: Single-Cycle vs Pipeline

- Single-Cycle Datapath

Instr	Data Path	Cycle Time	Total Time
R/I-type	IF+RR+ALU+RW	400+300+200+300	1200 ps
LW	IF+RR+ALU+MR+RW	400+300+200+400+300	1600 ps
SW	IF+RR+ALU+MW	400+300+200+400	1300 ps
BEQ	IF+RR+ALU	400+300+200	900 ps

Exercise: Single-Cycle vs Pipeline

- Pipeline Datapath

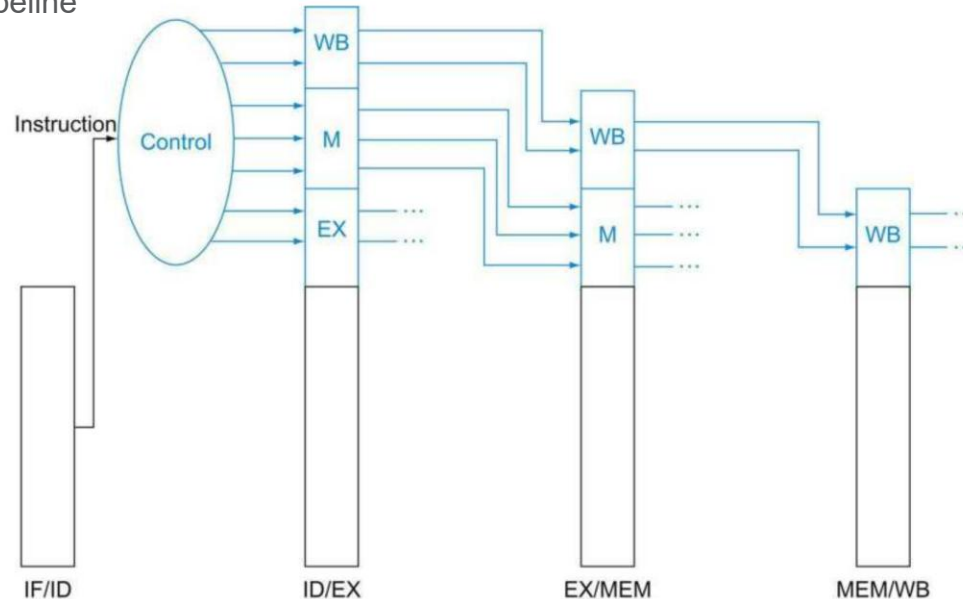
Instr	Data Path	Cycle Time	Total Time
R/I-type	IF RR ALU RW	Max(400,300,200,300)	400 ps
LW	IF RR ALU MR RW	Max(400,300,200,400,300)	400 ps
SW	IF RR ALU MW	Max(400,300,200,400)	400 ps
BEQ	IF RR ALU	Max(400,300,200)	400 ps



5-Stage Pipeline RISC-V Processor

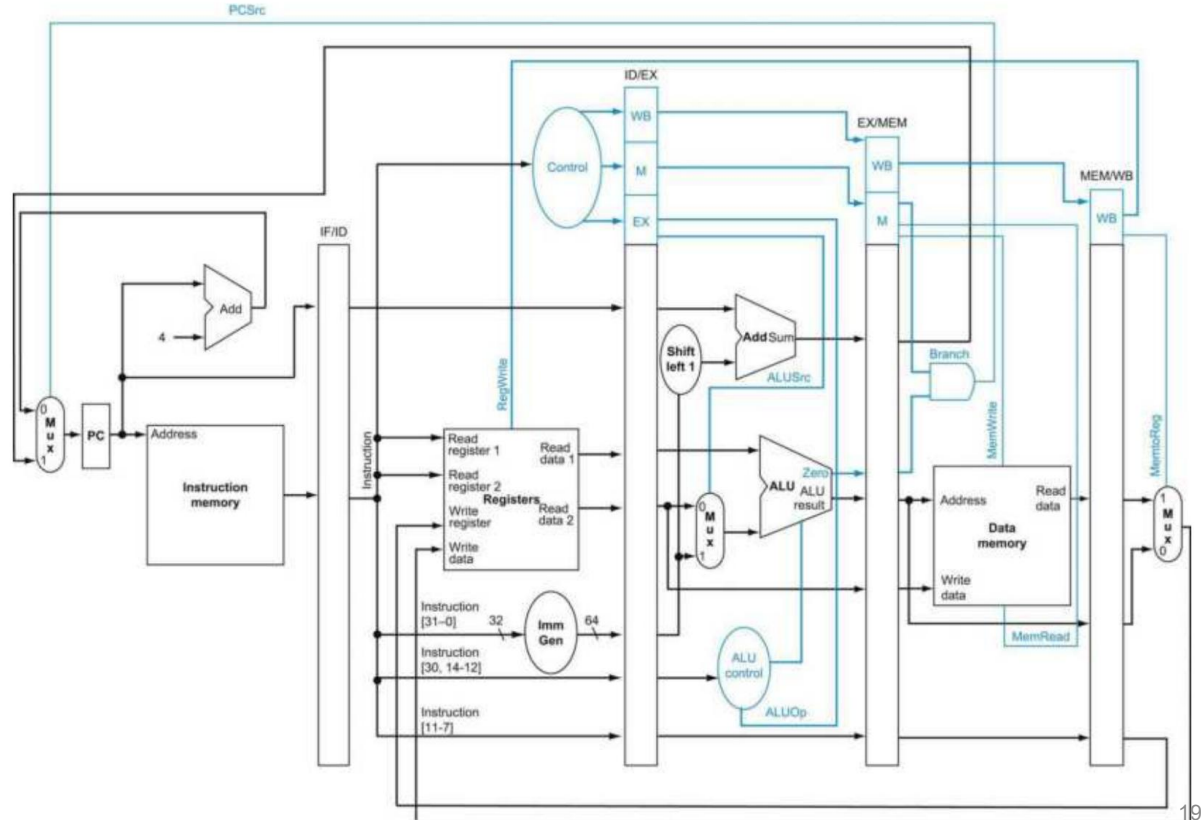
Pipelined control logic

- Propagate the flags along the pipeline



Basic RISC-V CPU Data & Control Path

- Data Path
 - 5 stages
 - Data connections
- Control Path
 - Control units
 - Control signals

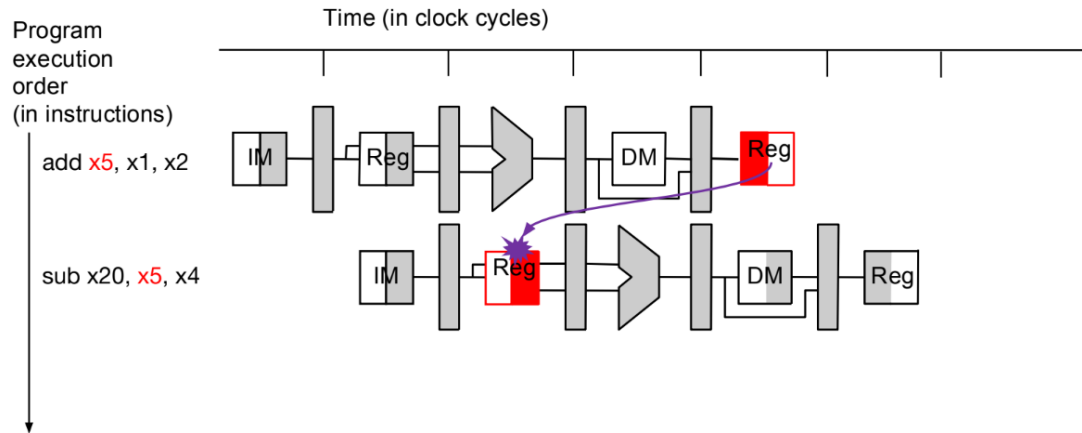


Pipeline Hazards

- Consider this scenario:
 - I0: add x5, x1, x2
 - I1: sub x20, x5, x4
- Source registers:
 - I0: x1, x2
 - I1: x5, x4
- Destination registers:
 - I0: x5
 - I1: x20

Pipeline Hazards

- Register dependency:
 - X5's result is not ready for I1
 - Should we care?



Pipeline Hazards

- Programmer expects instructions ordering:
 - To get expected result
 - I0: add x5, x1, x2
 - I1: sub x20, x5, x4

Pipeline Hazards

- Should the processor enforce instructions ordering?
 - Only externally
 - Memory is visible to S/W
 - **Registers are visible to S/W (e.g. debugger)**
 - Can do any optimization it wants internally

Pipeline Hazards

- What's a pipeline hazard?
 - **Resource sharing** conflicts inside the pipeline
 - Pipelining introduces resource sharing between instructions
 - Register file
 - Execution units
 - Instruction/data memory

Pipeline Hazards

- Types of pipeline hazard
 - Control hazard
 - Structural hazard
 - Data hazard

Control Hazards

- Control-flow dependency between instructions
 - Target address or condition not yet available to a branch instruction

Structural Hazards

- Resource sharing between instructions
 - Prior instruction using a resource that current instruction needs
 - e.g. Limited functional units
 - Register file ports limits
 - Shared instruction & data memory
 - Read-after-write memory
 - Synchronous read memory

Data Hazards

- Register sharing between instructions
 - Operands are not yet available to an instruction
 - Because prior instruction has not yet produced result

Data Hazards

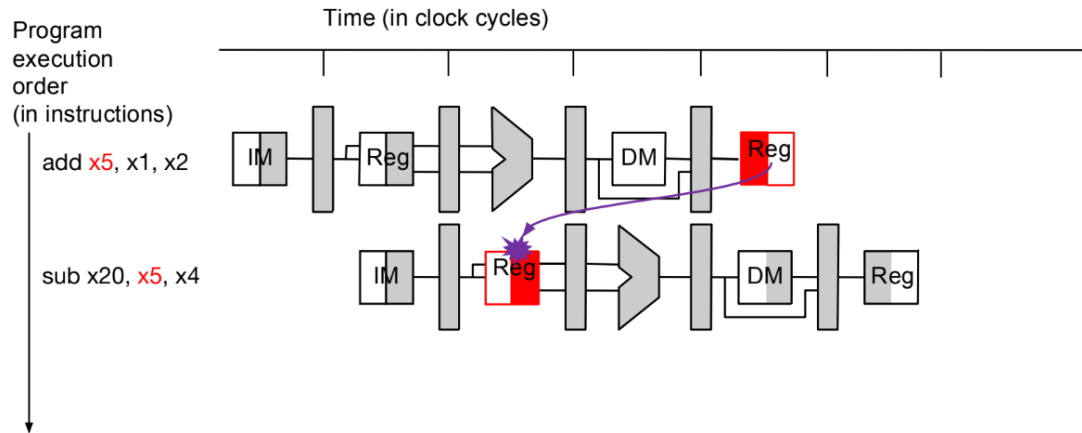
- Types of data Hazards
 - Read after write (RAW)
 - Write after write (WAW)
 - Write after read (WAR)
 - Read after read (RAR)

Data Hazards: RAW

- Reading a register before its write completes
- Example:
 - I0: add **x5**, x1, x2 // x5 is a destination register
 - I1: sub x20, **x5**, x4 // x5 is a source register

Data Hazards: RAW

- How many cycles should I1 stall?
 - I0: add **x5**, x1, x2
 - I1: sub x20, **x5**, x4

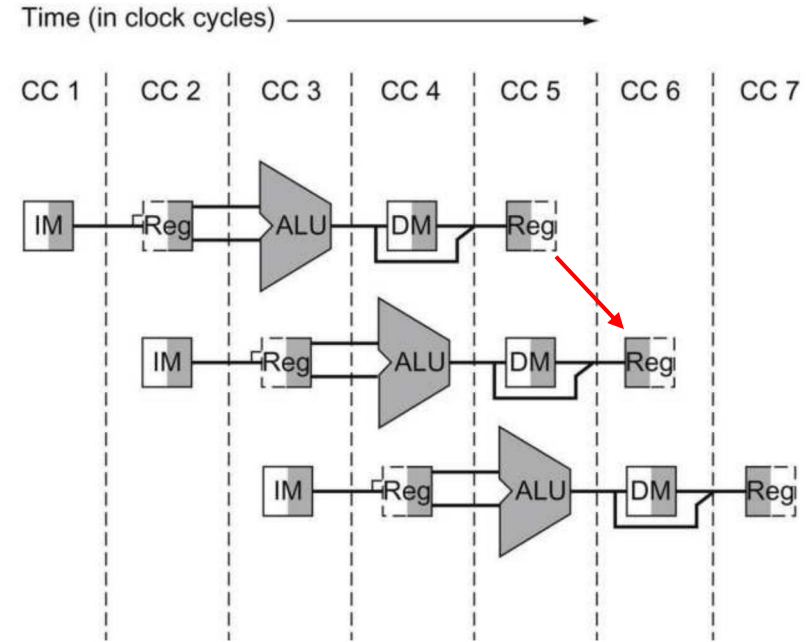
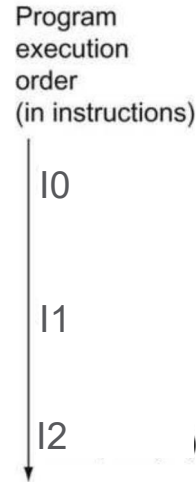


Data Hazards: WAW

- Writing to a register before its write completes
- Example:
 - I0: add **x5**, x1, x2
 - I1: sub **x5**, x3, x4
- Is it an issue in our current architecture?

Data Hazards: WAW

- I1 will update x5 after I0
 - I0: add **x5**, x1, x2
 - I1: sub **x5**, x3, x4



Data Hazards: WAW

- WAW is not an issue here
 - Due to in-order program execution
- Problematic if out-of-order commit is supported
 - e.g. variable-length execution stage

Whiteboard: ALU+MUL Extension

Example: We add a 1-cycle MUL unit into the EX stage

- ALU takes 0 cycle (100% combinational)
- MUL takes 1 cycle

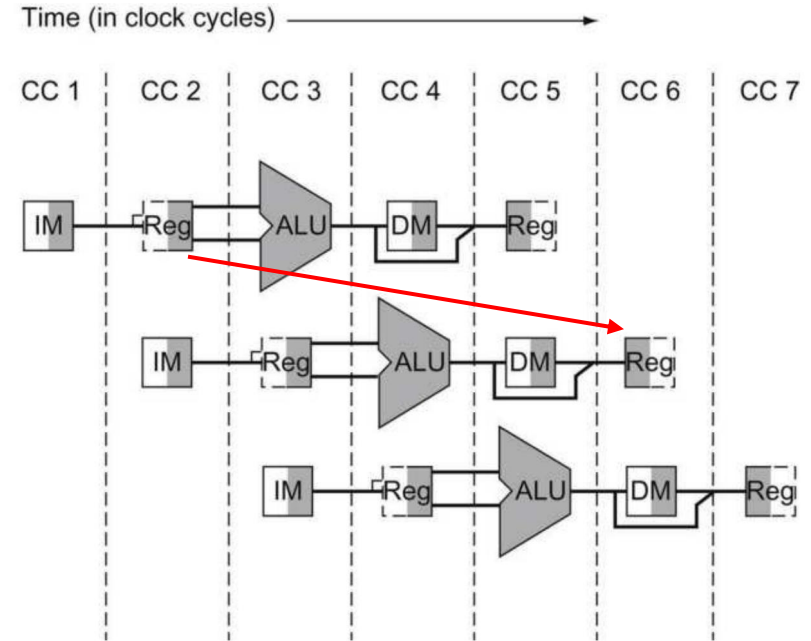
Data Hazards: WAR

- Writing to a register before it is read
- Example:
 - I0: add x2, x1, x5
 - I1: sub x5, x3, x4
- Is it a dependency issue?

Data Hazards: WAR

- I1 will update x5 after I0
 - I0: add x2, x1, **x5**
 - I1: sub **x5**, x3, x4

Program
execution
order
(in instructions)



Data Hazards: WAR

- WAR is not an issue here
 - Due to in-order program execution
- Problematic if out-of-order execution

Data Hazards: RAR

- Reading from a register before it is read
- Example:
 - I0: add x2, x1, x5
 - I1: sub x20, x5, x4
- Is it a dependency issue?

Data Hazards: RAR

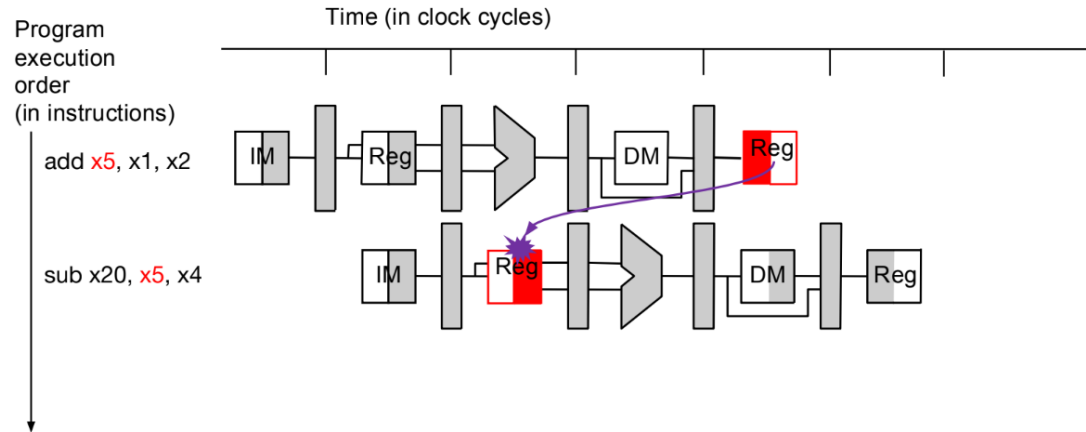
- No dependency issue
 - We are only reading

Resolving RAW hazards

- Naïve approach:
 - We stall the pipeline!
 - Wait until the register value is ready

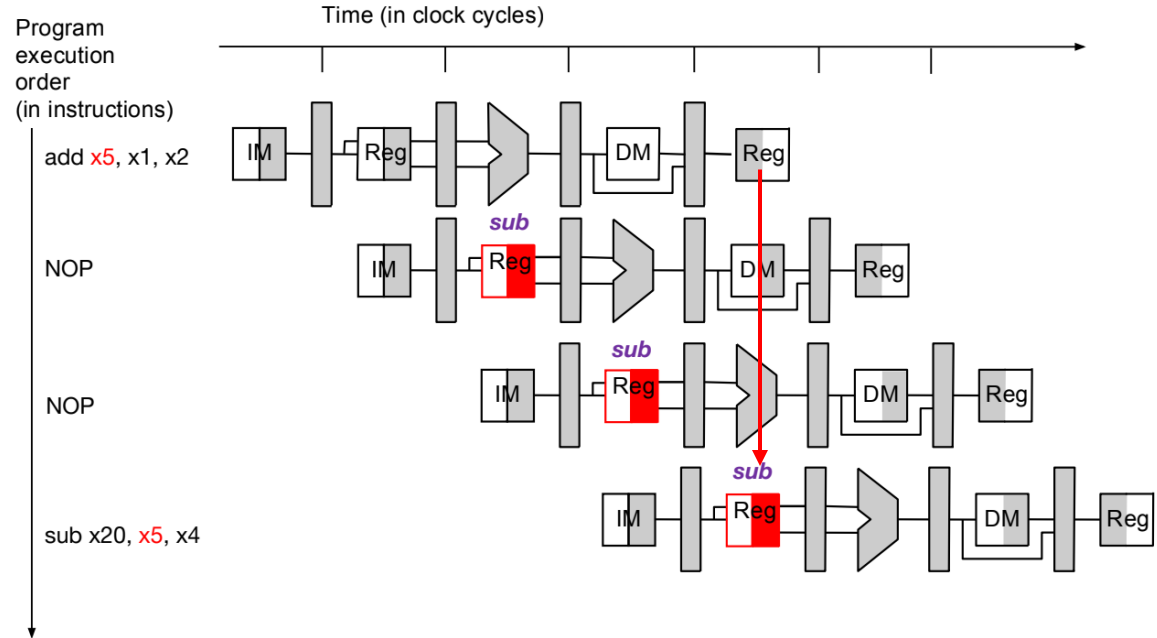
Resolving RAW hazards

- Example1:
 - I0: add **x5**, x1, x2
 - I1: sub x20, **x5**, x4



Resolving RAW hazards

- Solution1:
 - Insert 2 bubbles
 - Or 2 NOPs
- Assume Reg File R/W in same cycle

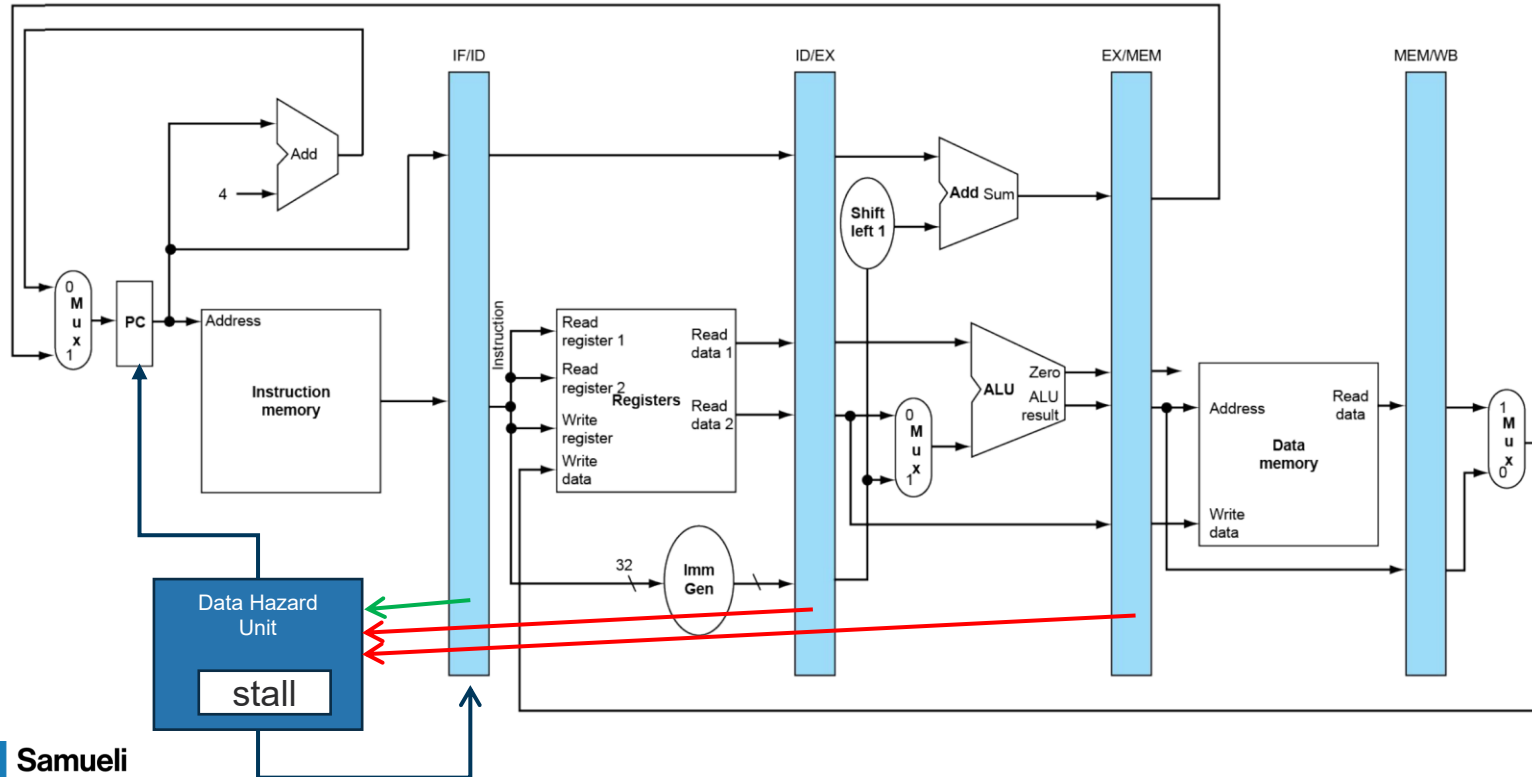


Attendance

- Please Fill the form with today's keyword to register your attendance.



RAW Hazard Implementation

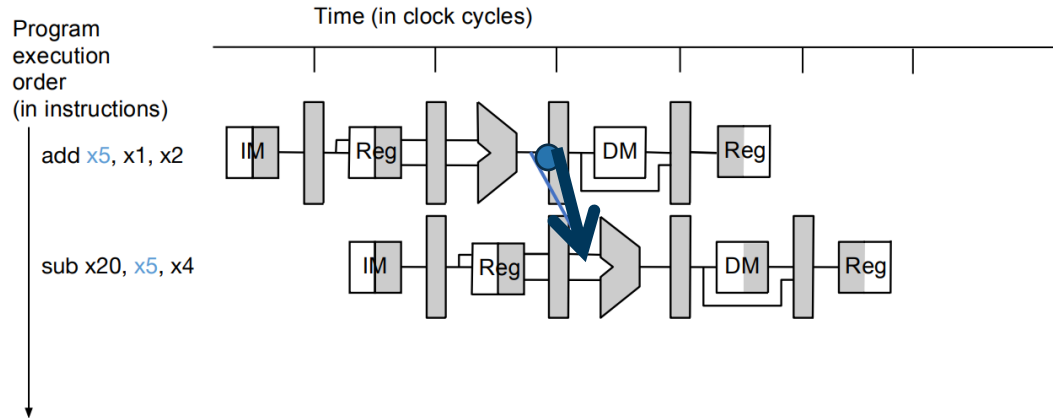


Resolving RAW hazards

- Any better solution?
 - Stalls hurt pipeline performance
 - How to prevent stalls?
 - Use Data Forwarding instead

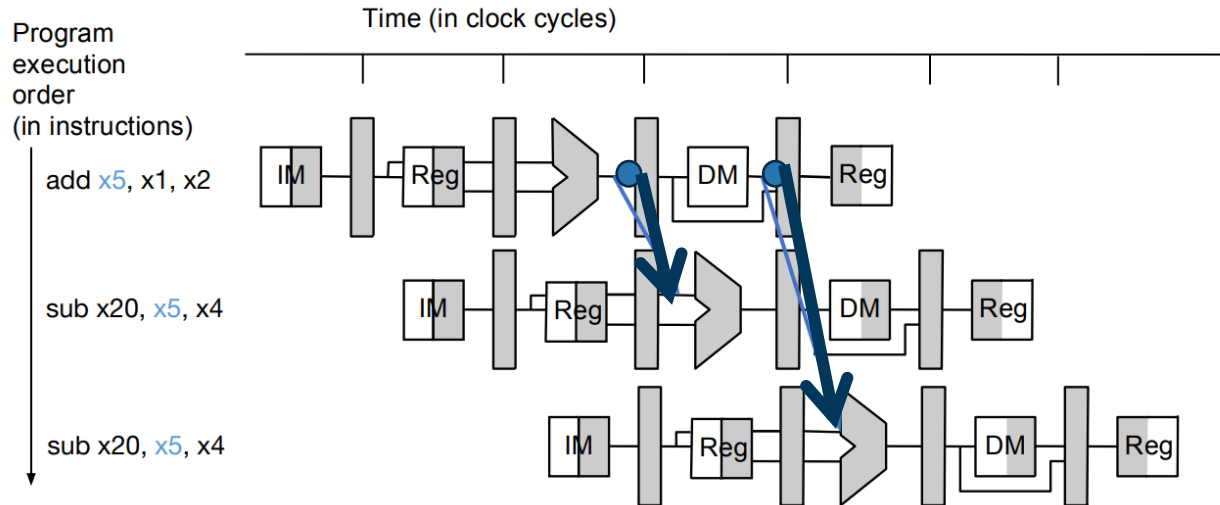
Resolving RAW hazards

- Data forwarding:
 - Redirect computed result directly to ALU
 - From EX/MEM to EX stage



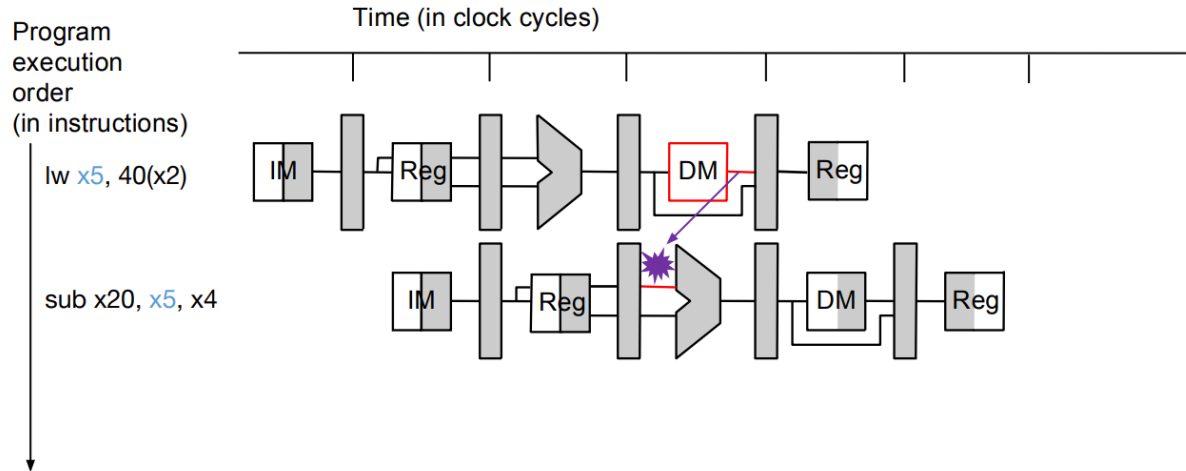
Resolving RAW hazards

- Data forwarding:
 - Long forwarding path
 - From MEM/WB to EX stage



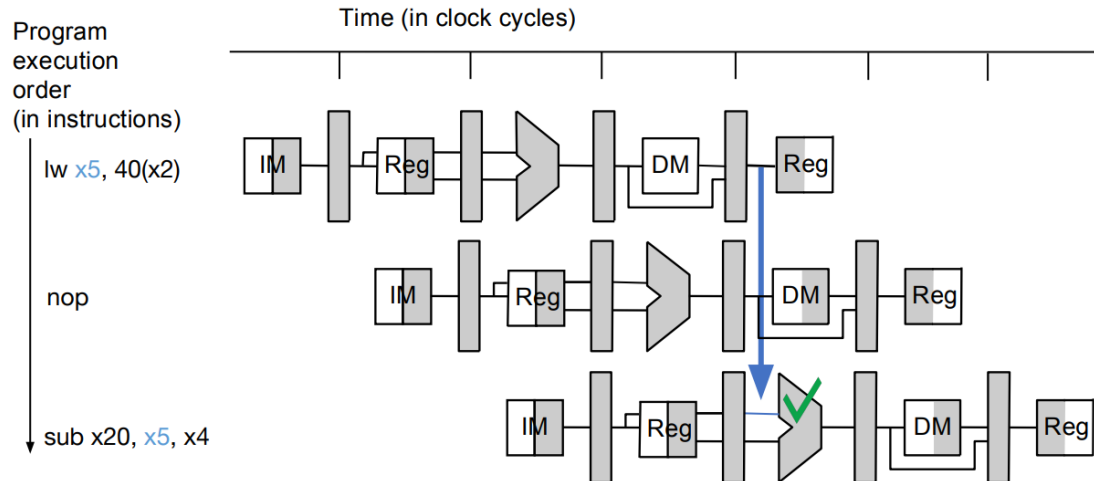
Resolving RAW hazards

- Data forwarding: Memory-ALU
 - Need direct forwarding from WB latch to EX stage
 - Not practical



Resolving RAW hazards

- Data forwarding: Memory-ALU
 - Add one bubble/NOP
 - Then forwarding from MEM/WB to EX stage

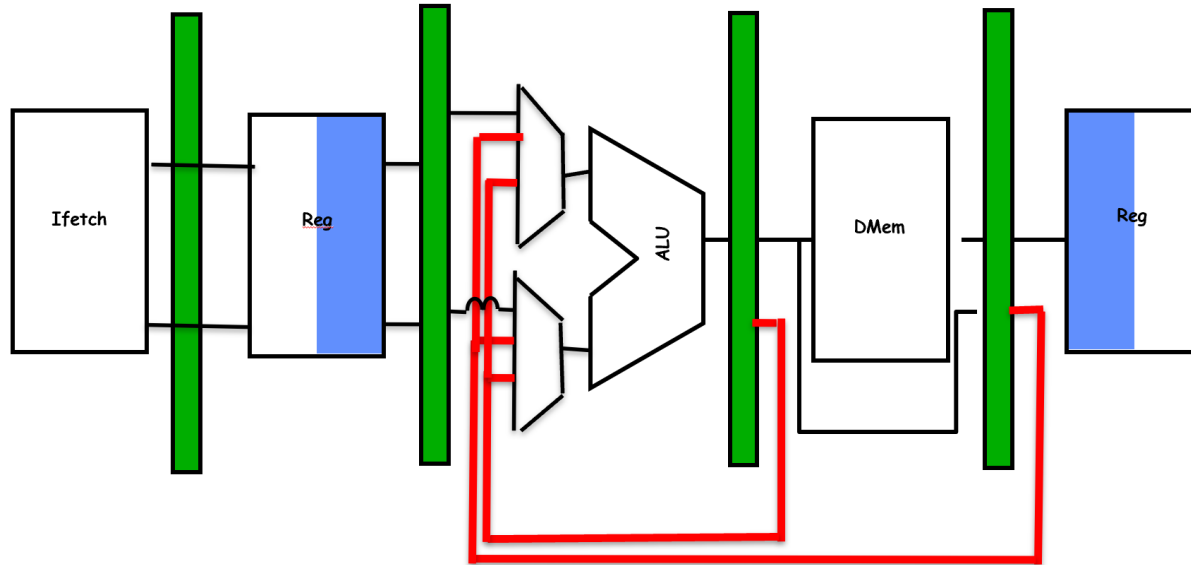


Resolving RAW hazards

- Data forwarding: Cost
 - Need extra wiring
 - Need multiplex to choose ALU source
 - Propagation delay may increase

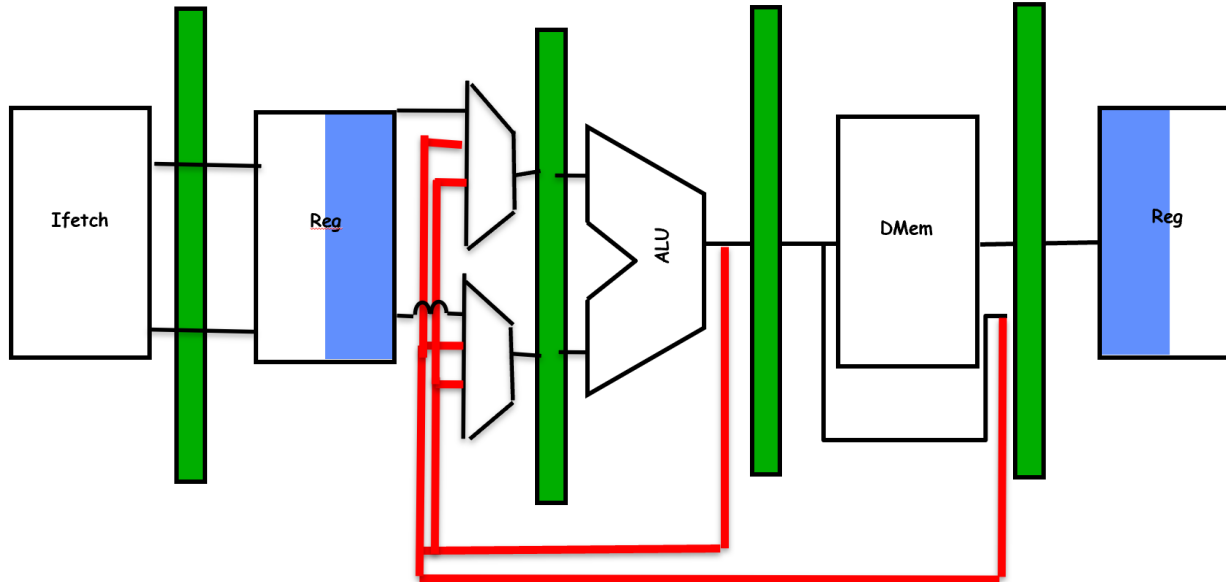
Resolving RAW hazards

- Data forwarding: design1
 - Insert MUXs before ALU



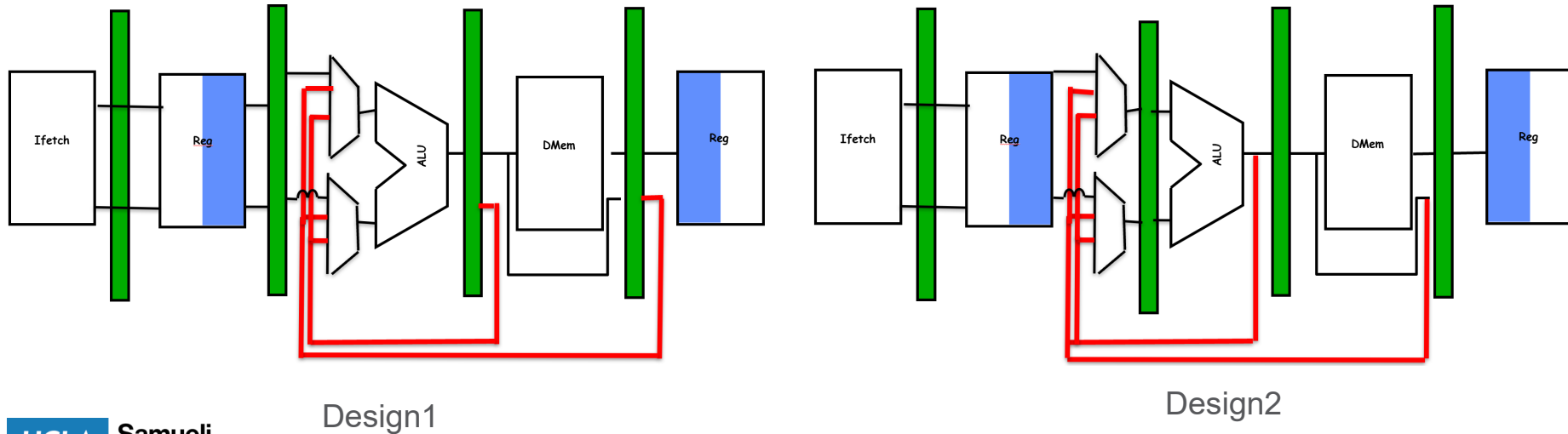
Resolving RAW hazards

- Data forwarding: design2
 - Insert MUXs before register



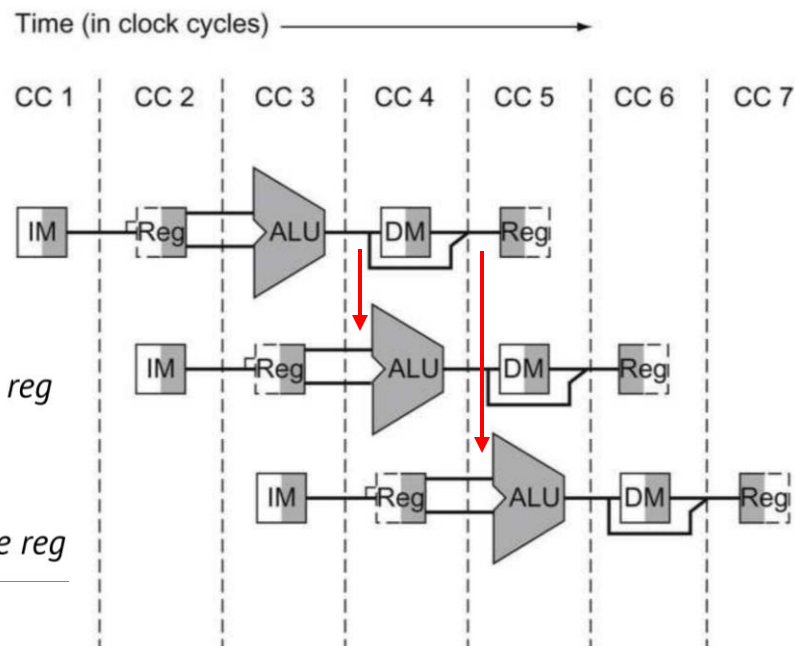
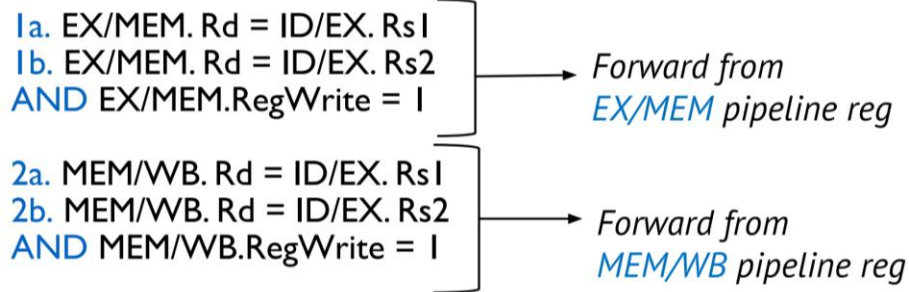
Resolving RAW hazards

- Data forwarding: which design is better?
 - The one with shorter propagation delay
 - Register File extension vs ALU extension
 - **Design1 is better**



Resolving RAW hazards

- Data forwarding: detect RAW forwarding?
 - $(rd == x0) \Rightarrow$ no write



Resolving RAW hazards

- Software solutions:
 - Use instruction scheduling to reorder instructions to minimize hazards

```
1. ld x2, 0(x1) // Load: x2 = Memory[x1 + 0]
2. add x3, x2, x4 // Add: x3 = x2 + x4, but x2 is not ready
3. sub x5, x6, x7 // Subtract: x5 = x6 - x7 (independent of the above)
```



```
1. ld x2, 0(x1) // Load: x2 = Memory[x1 + 0]
2. sub x5, x6, x7 // Subtract: x5 = x6 - x7 (independent of the load)
3. add x3, x2, x4 // Add: x3 = x2 + x4, x2 is now ready
```


Control Hazards

- Understanding branches
 - e.g. for loop

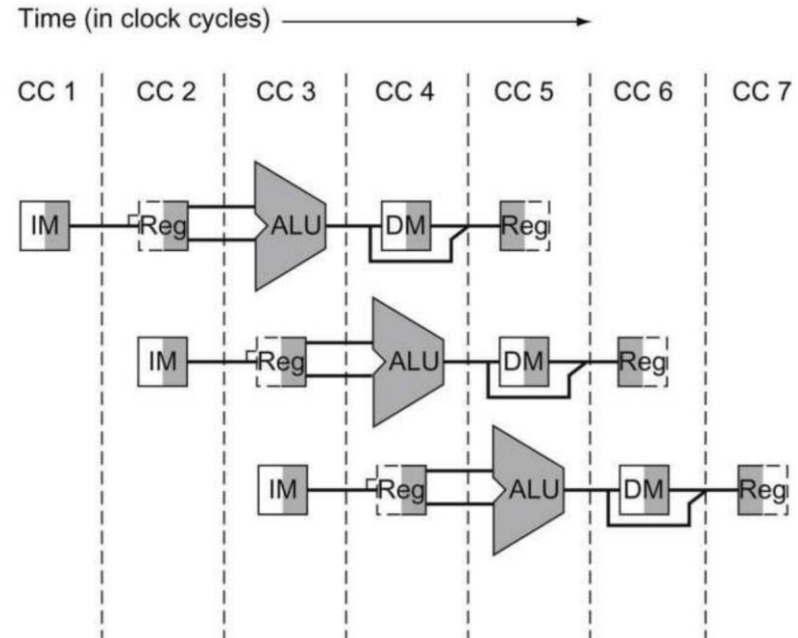
```
int main() {  
    for (int i = 0; i < 10; i++) {  
        // loop body  
    }  
    return 0;  
}
```



```
main:  
    li t0, 0           # i = 0  
    li t1, 10          # n=10  
loop:  
    addi t0, t0, 1      # i = i + 1  
    bne t0, t1, loop    # if (i != n) => loop  
exit:  
    li a0, 0           # return 0  
    ret
```

Control Hazards

- Data and control dependence
 - I0: addi x5, x5, 1
 - I1: bne x5, x6, loop
- Whether to branch?
 - (x5 == x6)
- Where to branch?
 - PC + imm



Control Hazards

RISC-V branches

- Conditional branches
 - BEQ r1, r2, imm; // if (r1 == r2) => PC = PC + imm
- Unconditional branches
 - JAL rd, imm; // rd = PC + 4; PC = PC + imm
 - JALR rd, rb, imm; // rd = PC + 4; PC = rb + imm

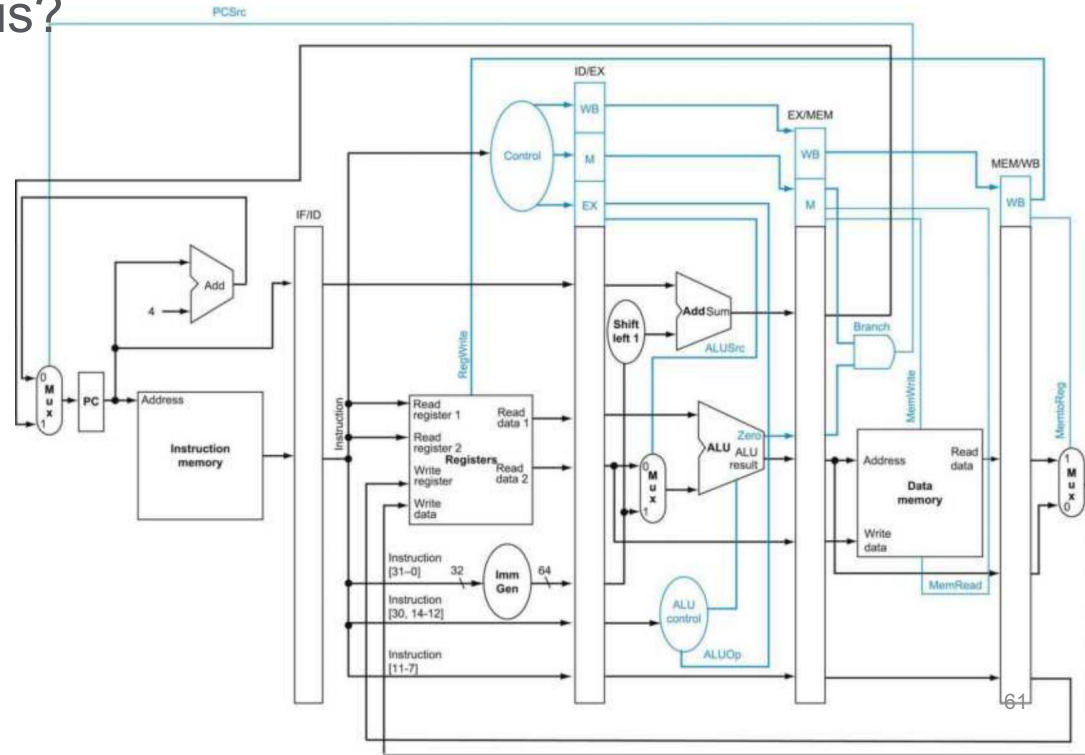
Control Hazards

CPU must answer 3 questions:

- Which type of branch?
- Whether to branch?
- Where to branch?

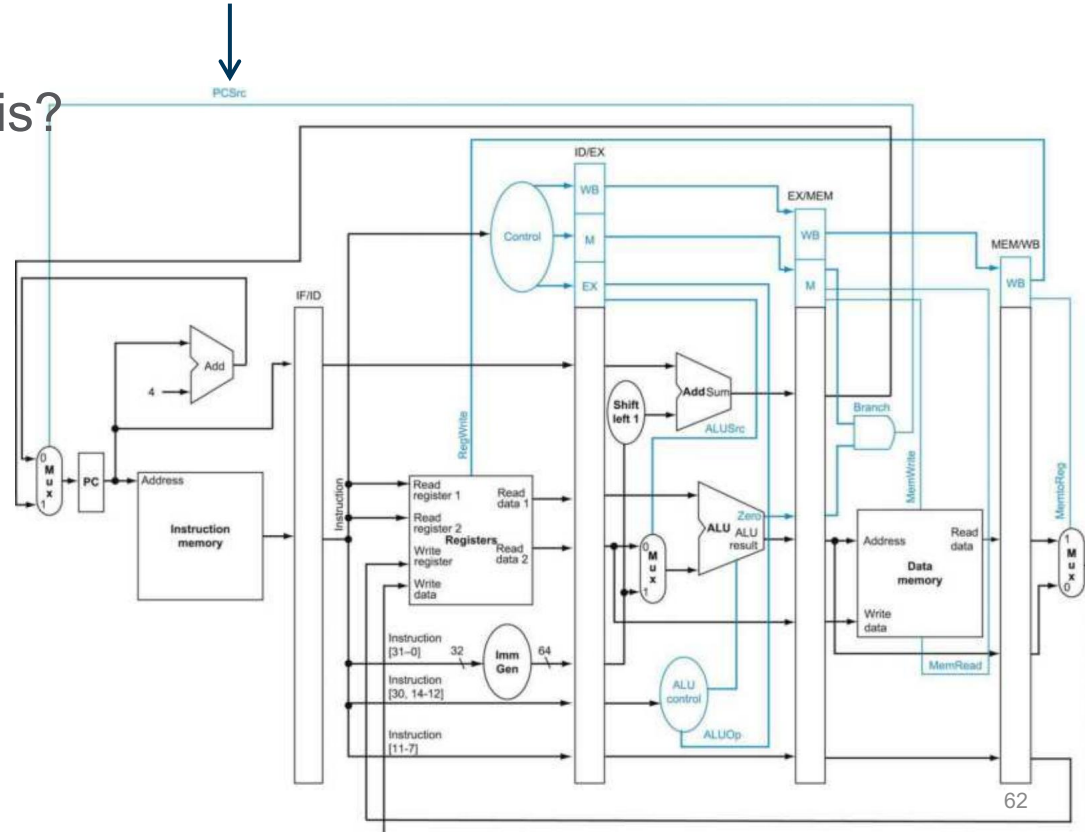
Control Hazards

- Which type of branch
 - Where to determine this?
 - ID stage



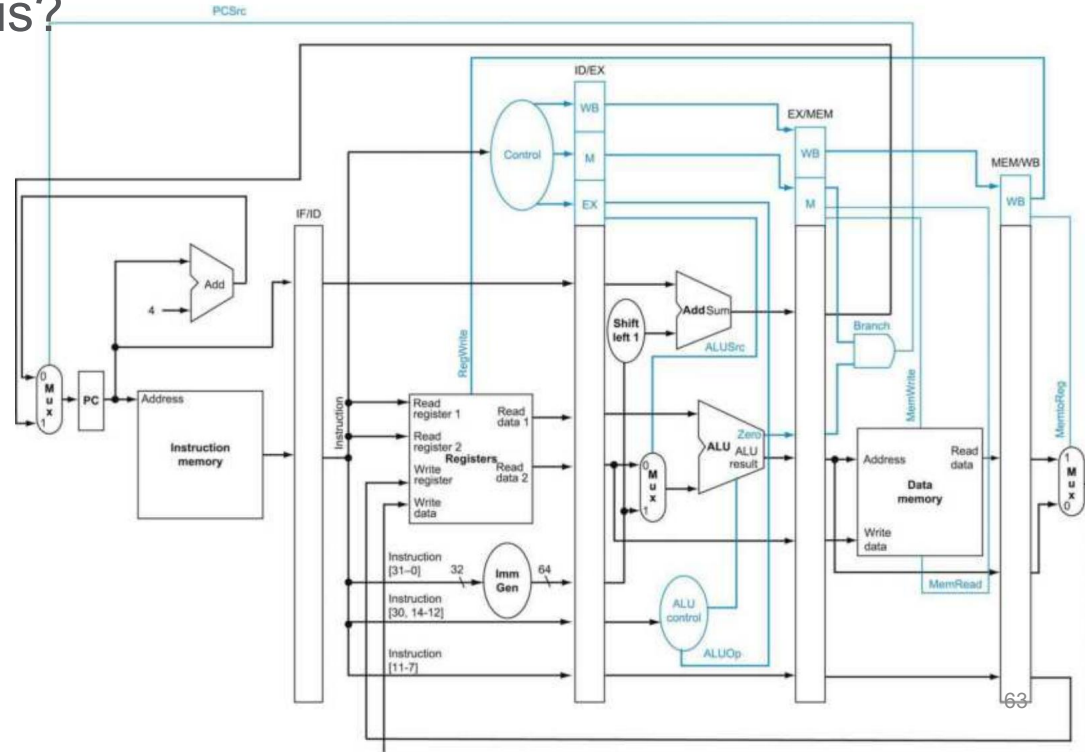
Control Hazards

- Whether to branch
 - Where to determine this?
 - MEM stage (PCSrc)



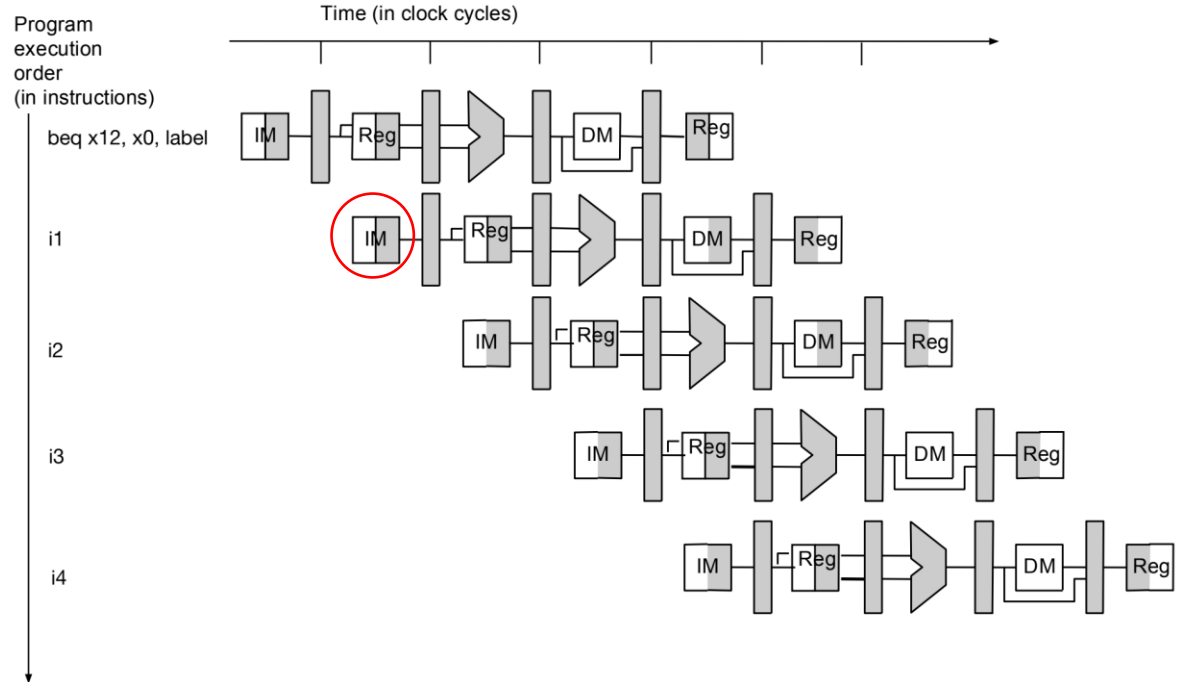
Control Hazards

- Where to branch
 - Where to determine this?
 - MEM stage



Control Hazards

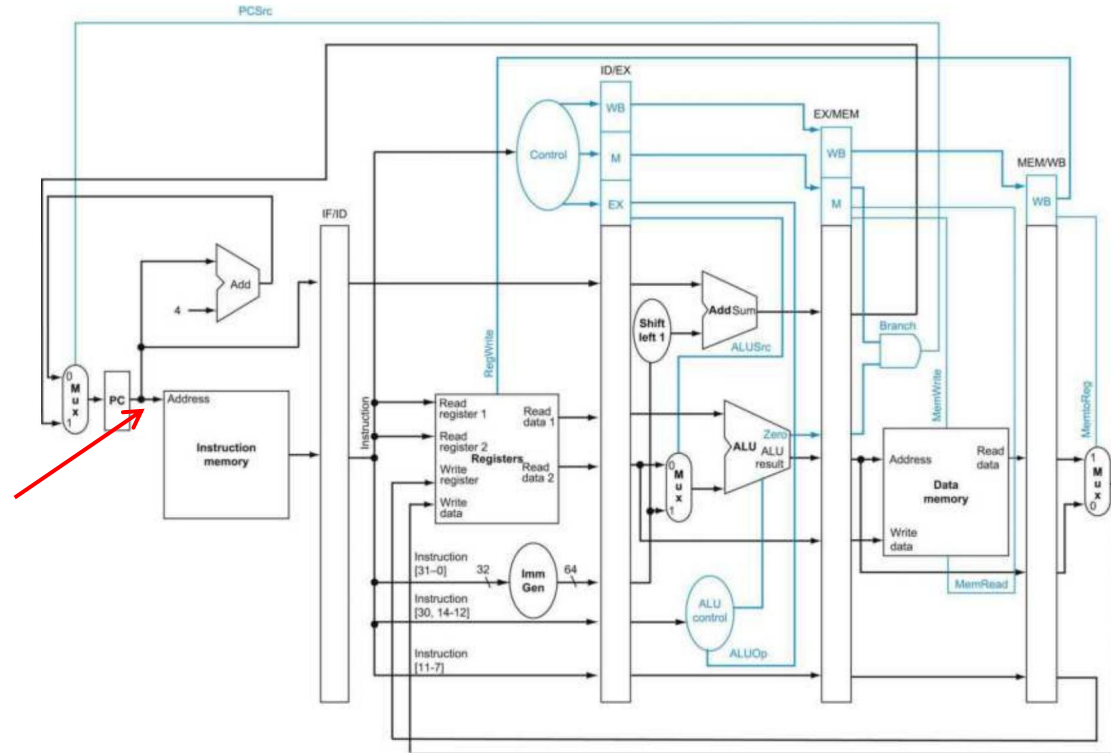
- Which stage should stall?
 - IF stage
- Why?
 - **Need new PC**



Control Hazards

- When is next PC available?
 - When PCSrc ready
 - When PC mux ready
 - MEM

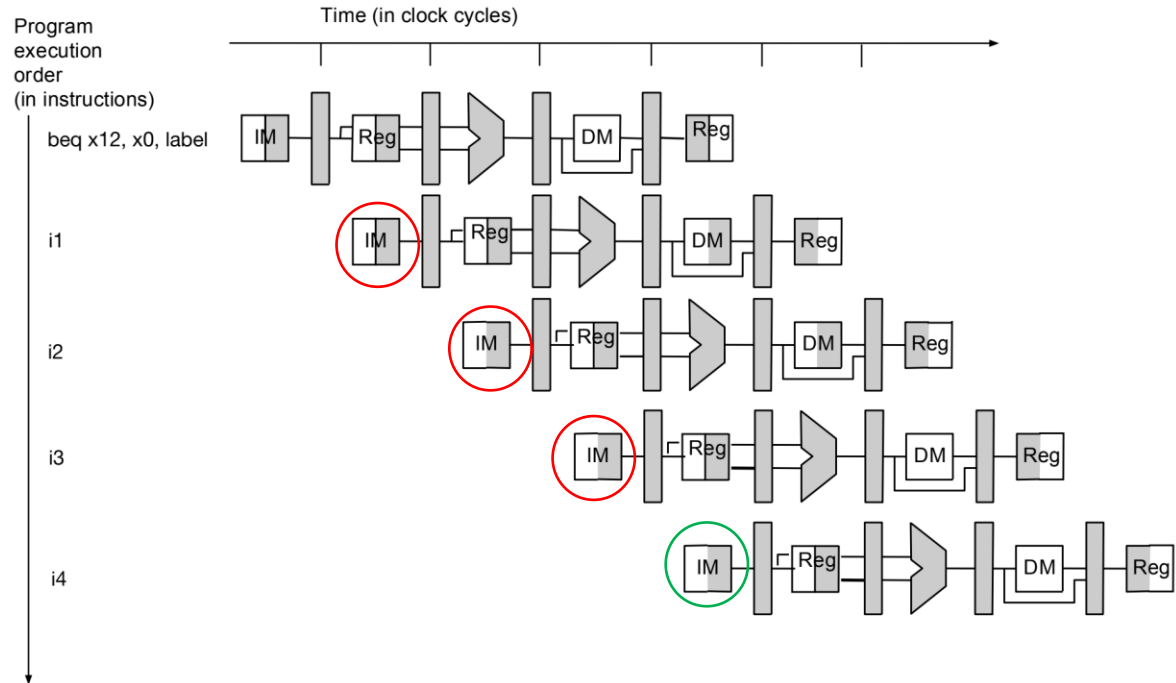
Assume IM access and PC update happen in same cycle, in IF stage



Control Hazards

- Stall cycles needed?
 - 3 cycles

Can we do better?



Control Hazards Handling

- Software Solution: Insert NOPs
 - S/W insert NOPs in delay slots
 - Pros:
 - No H/W cost
 - Cons:
 - Bad performance: still stalling
 - S/W depends on H/W stall count

Control Hazards Handling

- Software Solution: Delayed branch
 - S/W insert useful instructions in delay slots
 - Pros:
 - No H/W cost
 - Improved performance
 - Cons:
 - Hard to find useful instructions
 - S/W depends on H/W stall count

Control Hazards Handling

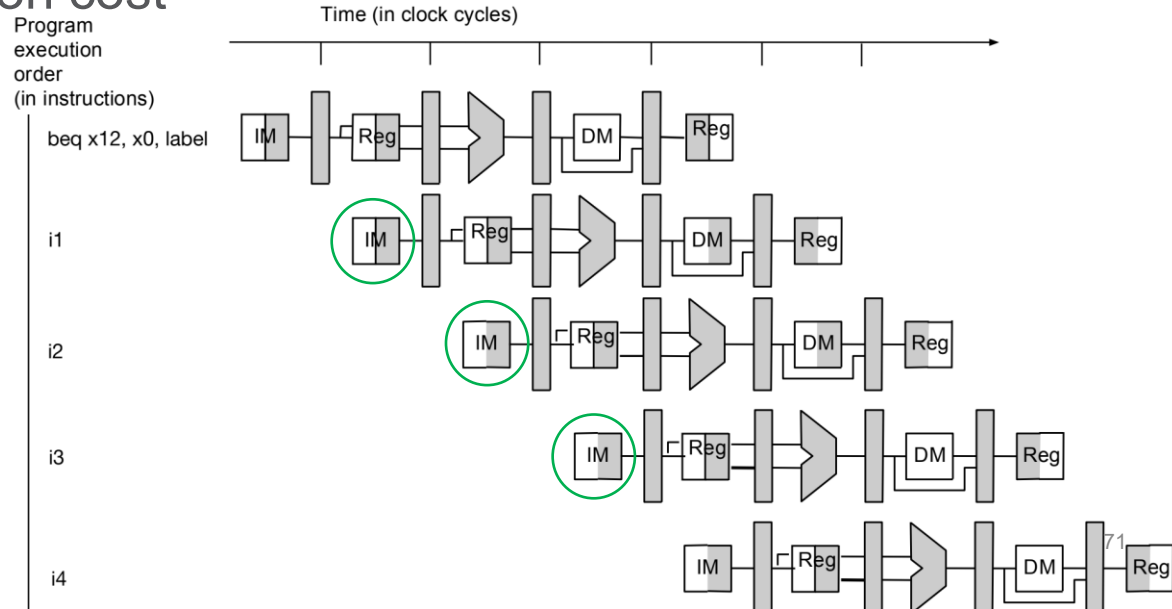
- Hardware Solution: Stall the pipeline
 - Pros:
 - S/W independence
 - Minimal H/W cost
 - Cons:
 - Bad performance

Control Hazards Handling

- Hardware Solution: Speculation
 - Pros:
 - S/W independence
 - Better performance
 - Cons:
 - H/W cost

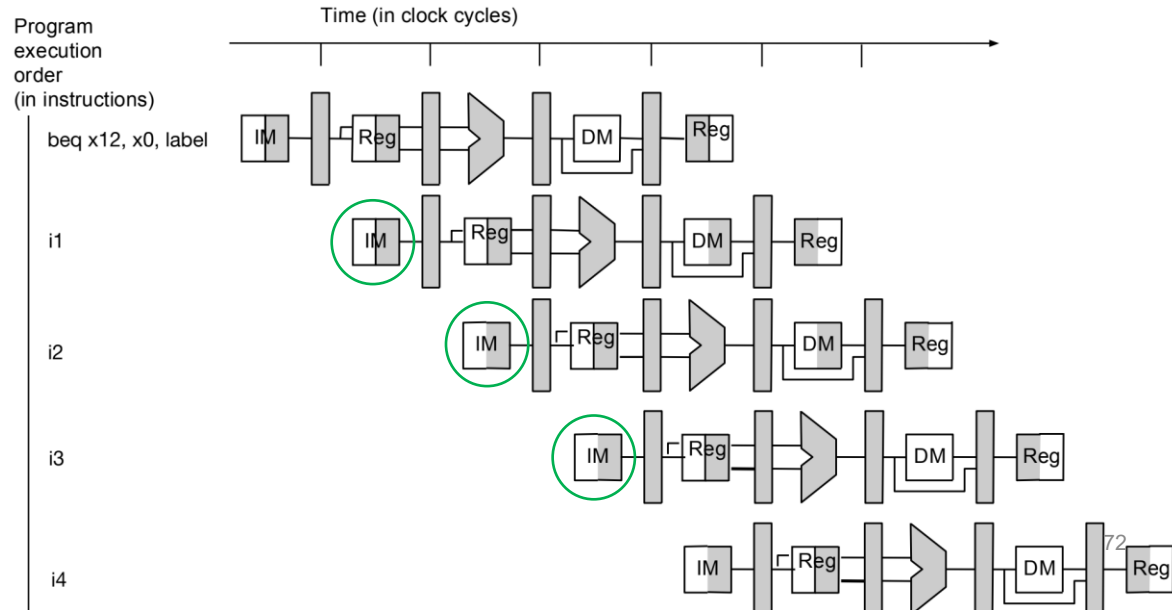
Control Hazards: Speculation

- Predict: Not taken
 - Emulate delay branch
 - Pros: no wasted cycles
 - Cons: mis-prediction cost



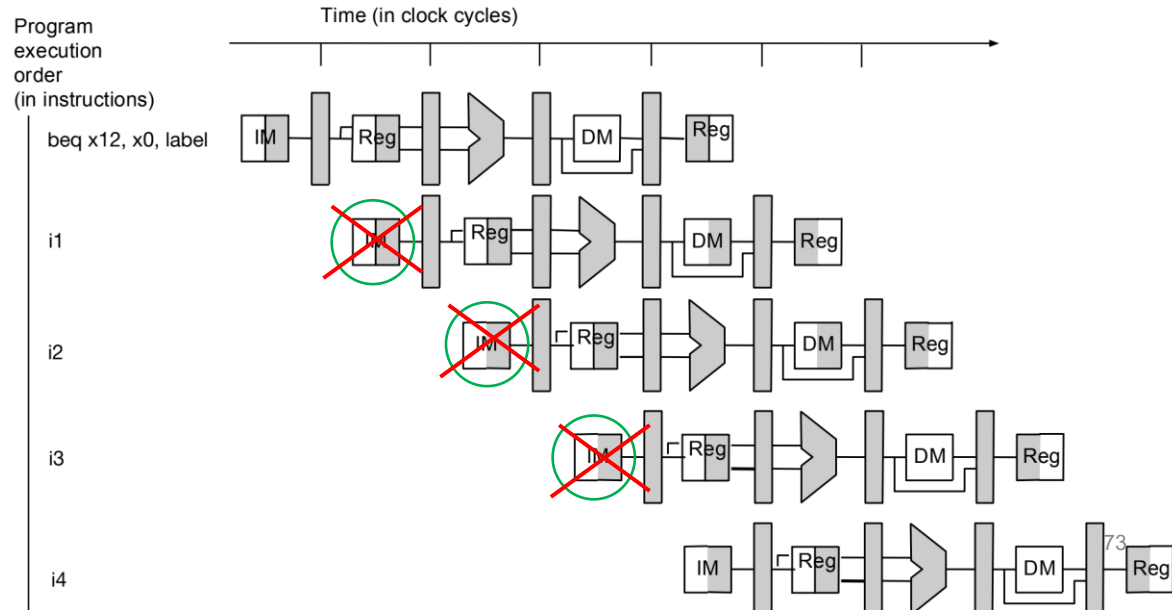
Control Hazards: Speculation

- Predict: Not taken
 - How to detect misprediction?
 - Branch condition is taken
 - When?
 - MEM stage



Control Hazards: Speculation

- Predict: Not taken
 - How to recover from misprediction?
 - Flush the pipeline (invalidate each previous stage)
 - Reset PC



Control Hazards: Speculation

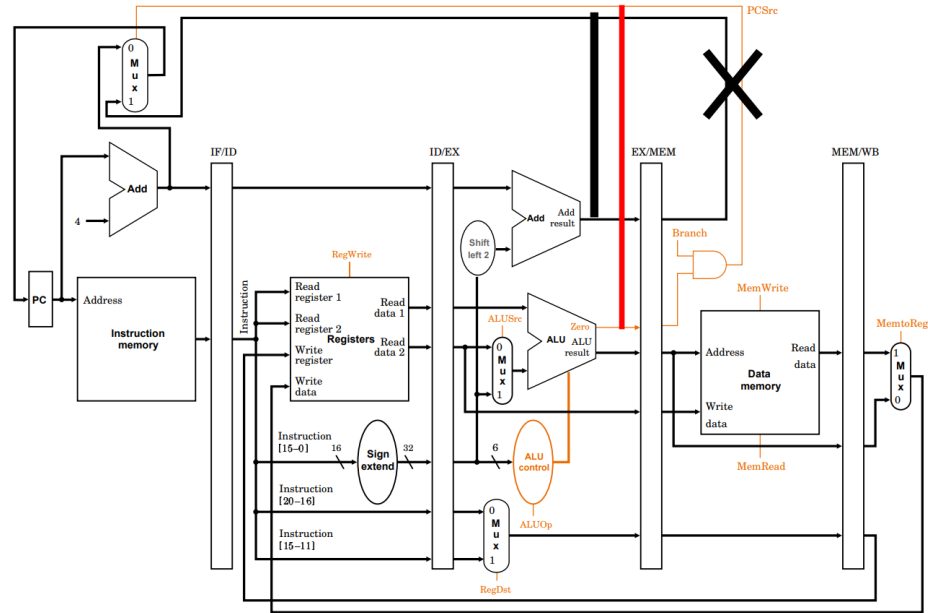
- Practical Branch stats
 - About 20% of instructions is control-flow
 - 50% of forward branches (if-then-else) are taken
 - 90% of backward branches (loops) are taken
 - 30% of all branches are not-taken

Control Hazards: Speculation

- Practical Branch stats
 - About 20% of instructions is control-flow
 - 50% of forward branches (if-then-else) are taken
 - 90% of backward branches (loops) are taken
 - 30% of all branches are not-taken
- Not-taken prediction accuracy?
 - Only 30%
 - Cost: 3 cycles

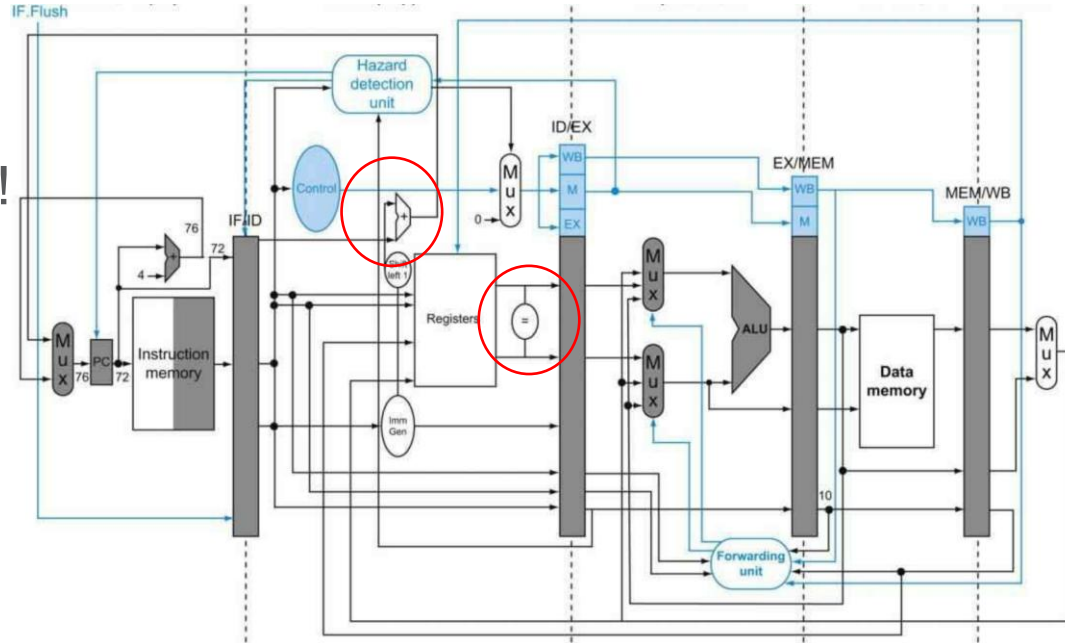
Control Hazards: Speculation

- Reducing Misprediction Cost
 - Resolving branches sooner
 - Making it 2 cycles can be done



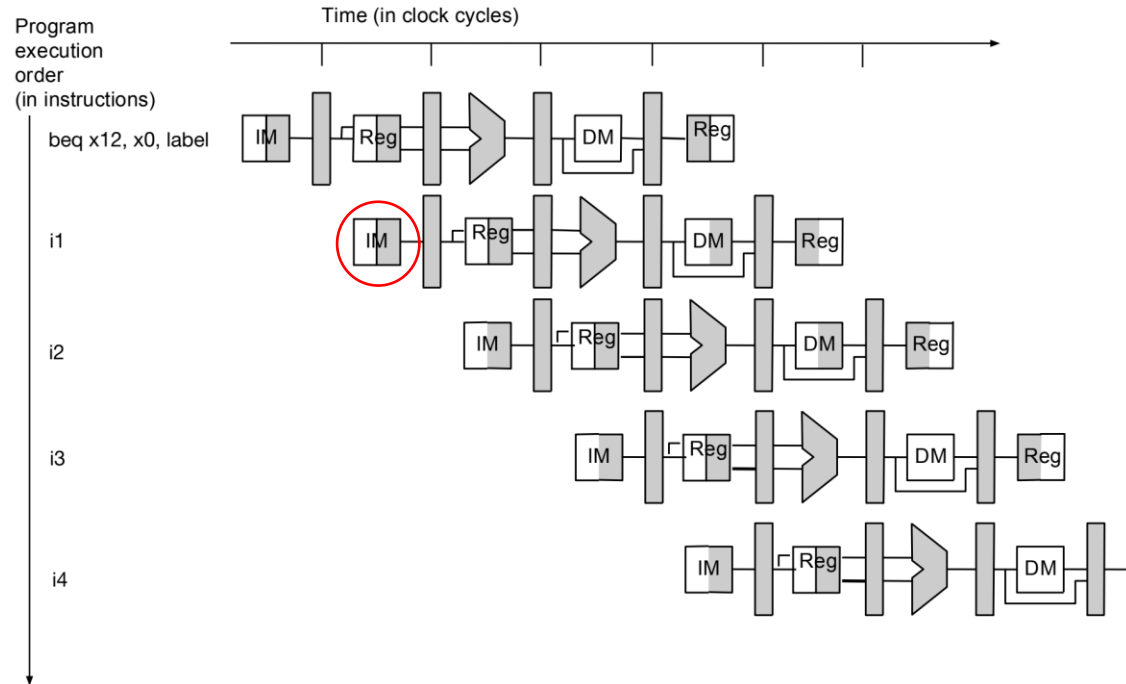
Control Hazards: Speculation

- Reducing Misprediction Cost
 - More aggressive solution: **Branch only**
 - Move NextPC to DE
 - Move PC + imm to DE
- Warning: r1/r2 RAW
 - Need data forwarding!



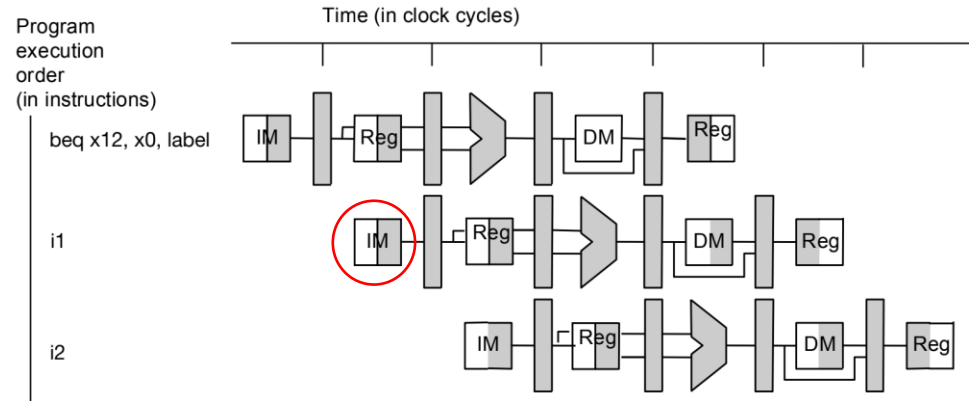
Control Hazards: Speculation

- Predict: Always Taken
- What do we need?
 - nextPC
 - At fetch stage



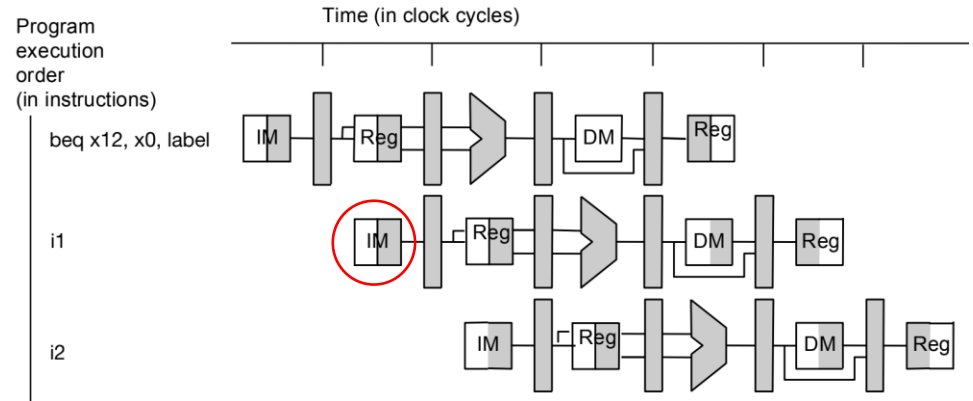
Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain nextPC?
 - We can predict



Control Hazards: Speculation

- Predict: Always Taken
 - How to obtain nextPC?
 - We can predict



Acknowledgement

- This course is partly inspired and use materials from the following courses created by my colleagues:
 - ECE-M1126C: Nader Sehatbakhsh @ UCLA
 - CS3220: Hyesoon Kim @ gatech

Q&A
